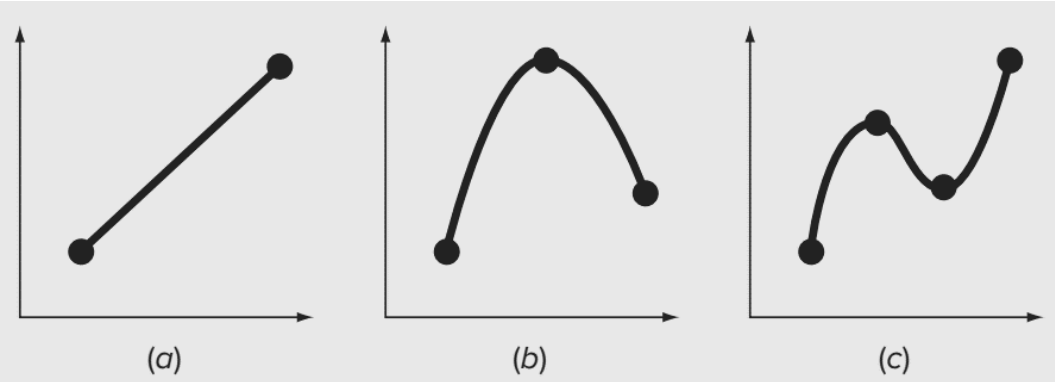
LAB 7: Polynomial Interpolation

Objective: To obtain the linear/polynomial interpolation using Newton/Lagrange's polynomial.

Theory: You will frequently have occasion to estimate intermediate values between precise data points. The most common method used for this purpose is polynomial interpolation. The general formula for an (n − 1)th-order polynomial can be written as-

$$f(x) = a_1 + a_2 x + a_3 x^2 + \cdots + a_n x^{n-1}$$

For n data points, there is one and only one polynomial of order (n − 1) that passes through all the points. For example, there is only one straight line (i.e., a first-order polynomial) that connects two points. Similarly, only one parabola connects a set of three points. Polynomial interpolation consists of determining the unique (n−1) order polynomial that fits n data points. This polynomial then provides a formula to compute intermediate values.



Examples of interpolating polynomials: (*a*) first-order (linear) connecting two points, (*b*) second-order (quadratic or parabolic) connecting three points, and (*c*) third-order (cubic) connecting four points.

LAGRANGE INTERPOLATING POLYNOMIAL

Suppose we formulate a linear interpolating polynomial as the weighted average of the two values that we are connecting by a straight line:

$$f(x) = L_1 f(x_1) + L_2 f(x_2) \tag{17.19}$$

where the L's are the weighting coefficients. It is logical that the first weighting coefficient is the straight line that is equal to 1 at x_1 and 0 at x_2:

$$L_1 = \frac{x - x_2}{x_1 - x_2}$$

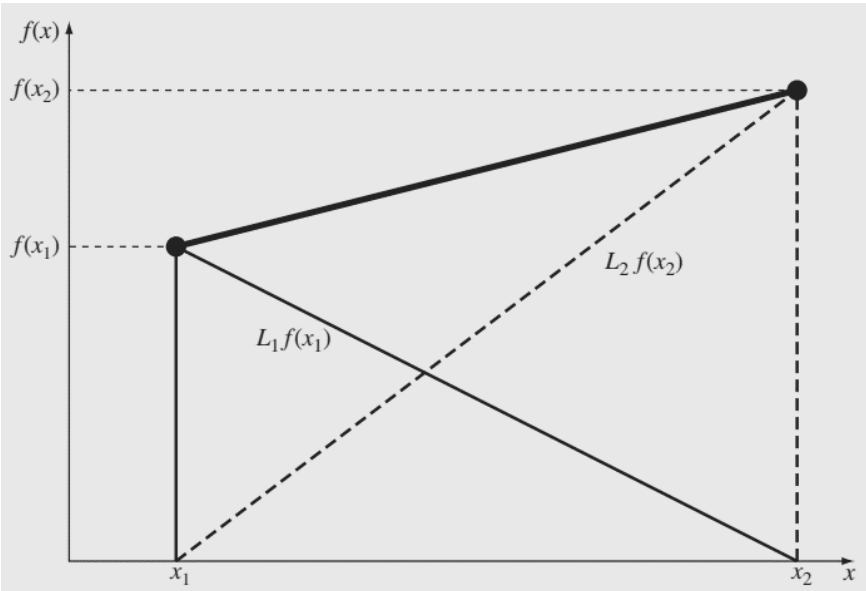
Similarly, the second coefficient is the straight line that is equal to 1 at x_2 and 0 at x_1:

$$L_2 = \frac{x - x_1}{x_2 - x_1}$$

Substituting these coefficients into Eq. (17.19) yields the straight line that connects the points (Fig. 17.8):

$$f_1(x) = \frac{x - x_2}{x_1 - x_2} f(x_1) + \frac{x - x_1}{x_2 - x_1} f(x_2) \tag{17.20}$$

where the nomenclature $f_1(x)$ designates that this is a first-order polynomial. Equation (17.20) is referred to as the *linear Lagrange interpolating polynomial.*



A visual depiction of the rationale behind Lagrange interpolating polynomials. The figure shows the first-order case. Each of the two terms of Eq. (17.20) passes through one of the points and is zero at the other. The summation of the two terms must, therefore, be the unique straight line that connects the two points.

The same strategy can be employed to fit a parabola through three points. For this case three parabolas would be used with each one passing through one of the points and equaling zero at the other two. Their sum would then represent the unique parabola that connects the three points. Such a second-order Lagrange interpolating polynomial can be written as

$$f_2(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)} f(x_1) + \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)} f(x_2)$$
$$+ \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)} f(x_3) \tag{17.21}$$

Notice how the first term is equal to $f(x_1)$ at x_1 and is equal to zero at x_2 and x_3. The other terms work in a similar fashion.

where

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

where n = the number of data points and $\prod$ designates the "product of."

Problem Statement. Use a Lagrange interpolating polynomial of the first and second order to evaluate the density of unused motor oil at $T = 15$ °C based on the following data:

$x_1 = 0 \qquad f(x_1) = 3.85$

$x_2 = 20 \qquad f(x_2) = 0.800$

$x_3 = 40 \qquad f(x_3) = 0.212$

Solution. The first-order polynomial [Eq. (17.20)] can be used to obtain the estimate at $x = 15$:

$$f_1(x) = \frac{15 - 20}{0 - 20} 3.85 + \frac{15 - 0}{20 - 0} 0.800 = 1.5625$$

In a similar fashion, the second-order polynomial is developed as [Eq. (17.21)]

$$f_2(x) = \frac{(15 - 20)(15 - 40)}{(0 - 20)(0 - 40)} 3.85 + \frac{(15 - 0)(15 - 40)}{(20 - 0)(20 - 40)} 0.800$$
$$+ \frac{(15 - 0)(15 - 20)}{(40 - 0)(40 - 20)} 0.212 = 1.3316875$$

Assignment -

One numerical as suggested by Faculty.

Sample numerical.

Given the data

x	1	2	2.5	3	4	5
$f(x)$	0	5	7	6.5	2	0

Calculate f Lagrange polynomial interpolating polynomials of order 1 through 3. Choose the sequence of the points for your estimates to attain the best possible accuracy. That is, the points should be centered around and as close as possible to the unknown.

1st Order:

$$f(x) = f(x_0)\frac{(x-x_1)}{(x_0-x_1)} + f(x_1)\frac{(x-x_0)}{(x_1-x_0)}$$

2nd order:

$$f(x) = f(x_0)\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} + f(x_1)\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}$$

$$+ f(x_2)\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}$$

3rd Order:

$$f(x) = \sum_{i=0}^{3} f(x_i) \prod_{\substack{j=0 \\ j\neq i}}^{3} \frac{x-x_5}{x_i x_j}$$

Assumption and boundary condition:

[$f(1) = 0$ $f(6) = 0$ is true and fix]

Interpolating the values in the mid.

first order interpolation:

expsevenlinear.m * × expsevensecorder.m × expseventhirdorder.m × +

C:\Users\setud\OneDrive\Documents\MATLAB\expsevenlinear.m

```
x = [1 2 2.5 3 4 5];
F = [0 5 7 6.5 2 0];


for i = 2:5

    L1 = (x(i)-x(i+1))/(x(i-1)-x(i+1));
    L2 = (x(i)-x(i-1))/(x(i+1)-x(i-1));

    Fr(i) = L1 * F(i-1) + L2 * F(i+1);

    fprintf('Value for Fx at %.2f is %.2f\n', x(i), Fr(i));
end
```

Second order interpolation:

expsevensecorder.m * × expseventhirdorder.m × +

:\Users\setud\OneDrive\Documents\MATLAB\expsevensecorder.m

```
x = [1 2 2.5 3 4 5];
f = [0 5 7 6.5 2 0];

for i = 2:5

    if i < 4

        f0 = f(1) * (x(i)-x(i+1)) * (x(i)-x(5)) / ((x(1)-x(i+1)) * (x(1)-x(5)));
        f1 = f(i+1) * (x(i)-x(1)) * (x(i)-x(5)) / ((x(i+1)-x(1)) * (x(i+1)-x(5)));
        f2 = f(5) * (x(i)-x(1)) * (x(i)-x(i+1)) / ((x(5)-x(1)) * (x(5)-x(i+1)));

    else

        f0 = f(1) * (x(i)-x(i-1)) * (x(i)-x(5)) / ((x(1)-x(i-1)) * (x(1)-x(5)));
        f1 = f(i-1) * (x(i)-x(1)) * (x(i)-x(5)) / ((x(i-1)-x(1)) * (x(i-1)-x(5)));
        f2 = f(5) * (x(i)-x(1)) * (x(i)-x(i-1)) / ((x(5)-x(1)) * (x(5)-x(i-1)));

    end

    F = f0 + f1 + f2;

    fprintf('value for fx at %.2f is %.4f\n', x(i), F);

end
```

Third order interpolation

expseventhirdorder.m *

:\Users\setud\OneDrive\Documents\MATLAB\expseventhirdorder.m

```
x = [1 2 2.5 3 4 5];
f = [0 5 7 6.5 2 0];

for i = 2:5
    if i == 2
        idx = [1 3 4 5];
    elseif i == 3
        idx = [1 2 4 5];
    else
        idx = [1 i-1 i-2 5];

    end

    x0 = x(idx(1)); f0v = f(idx(1));
    x1 = x(idx(2)); f1v = f(idx(2));
    x2 = x(idx(3)); f2v = f(idx(3));
    x3 = x(idx(4)); f3v = f(idx(4));

    L0 = (x(i)-x1)*(x(i)-x2)*(x(i)-x3)/((x0-x1)*(x0-x2)*(x0-x3));
    L1 = (x(i)-x0)*(x(i)-x2)*(x(i)-x3)/((x1-x0)*(x1-x2)*(x1-x3));
    L2 = (x(i)-x0)*(x(i)-x1)*(x(i)-x3)/((x2-x0)*(x2-x1)*(x2-x3));
    L3 = (x(i)-x0)*(x(i)-x1)*(x(i)-x2)/((x3-x0)*(x3-x1)*(x3-x2));
    F = f0v*L0 + f1v*L1 + f2v*L2 + f3v*L3;
    fprintf('value for fx at %.2f is %.4f\n', x(i), F);
end
```

For the taken previous condition the value become "0" for which we changed condition

Observation:

In the 3rd order the pattern was unusual, in this i saw that by going through previous pattern we got false value due to intersecting points forming 0/0 or ∞ as a value.

Results:

1st order:

```
avoid a potential name conflict
>> expsevenlinear
Value for Fx at 2.00 is 4.67
Value for Fx at 2.50 is 5.75
Value for Fx at 3.00 is 5.33
Value for Fx at 4.00 is 3.25
```

2nd Order:

```
>> expsevensecorder
value for fx at 2.00 is 6.0000
value for fx at 2.50 is 6.8125
value for fx at 3.00 is 6.6667
value for fx at 4.00 is 2.0000
```

3rd Order:

```
>> expseventhirdorder
value for fx at 2.00 is 6.1667
value for fx at 2.50 is 6.3438
value for fx at 3.00 is 7.6667
value for fx at 4.00 is 2.0000
```

Order of S(x) in different interpolation:

X	1	2	2.5	3	4	5
F(X)	0	5	7	6.5	2	0
First order	0	4.67	5.75	5.33	3.25	0
Second order	0	6	6.81	6.66	2	0
Third order	0	6.16	6.34	7.66	2	0